Lab Assignment-1

Name:	**Aalok Kumar Singh**
Roll_no:	**2401201185**
Course_Name/Code:	**Operating System Lab ENCA252**
Program_name:	**BCA (AI & DS)**
Assistant Professor:	**Dr. Sneh Lata Yadav**

Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

Introduction

CPU scheduling is an important function of an operating system. It determines the order in which processes are executed by the CPU. In this assignment, we implement and analyze two scheduling algorithms: First Come First Serve (FCFS) and Shortest Job First (SJF).

Objectives

- Understand CPU scheduling concepts
- Implement FCFS and SJF algorithms
- Calculate CT, TAT, WT
- Compare performance of algorithms **Tools which are used**
- Python 3.14.2
- VS Code / Command Promt
- Windows OS

Task 1: Process Creation & Input Handling

Description:-

In this task, we created a Process class with attributes Process ID (PID), Arrival Time (AT), and Burst Time (BT). We took input for multiple processes from the user and stored them in a list. The process details were then displayed in tabular format.

Code:-

```
# CPU Scheduling: FCFS & SJF (Non-Preemptive)
 class Process:     def
__init__(self, pid, at, bt):
        self.pid =
pid         self.at =
at         self.bt =
bt         self.ct =
0         self.tat =
0         self.wt = 0

# -------------------- INPUT --------------------
processes = []
 n = int(input("Enter number of processes:
"))
 for i in
range(n):
    print(f"\nEnter details for Process
{i+1}")     pid = input("Process ID: ")     at
= int(input("Arrival Time: "))     bt =
int(input("Burst Time: "))
processes.append(Process(pid, at, bt))

# -------------------- DISPLAY INPUT -------------------
- print("\nProcess Table:") print("PID\tAT\tBT") for p
in processes:
    print(f"{p.pid}\t{p.at}\t{p.bt}")
```

Screenshot:-

```
C:\Users\AALOK\Desktop\OS_ASSIGNMENT-1>python main.py
Enter number of processes: 3

Enter details for Process 1
Process ID: P1
Arrival Time: 0
Burst Time: 5

Enter details for Process 2
Process ID: P2
Arrival Time: 1
Burst Time: 3

Enter details for Process 3
Process ID: P3
Arrival Time: 2
Burst Time: 8

Process Table:
PID     AT      BT
P1      0       5
P2      1       3
P3      2       8
```

Task 2: FCFS Scheduling

Description:-

First Come First Serve (FCFS) is a non-preemptive scheduling algorithm in which the process that arrives first is executed first. Processes are executed in the order of their arrival time.

Steps:-

1. Processes are sorted based on Arrival Time.
2. Completion Time (CT) is calculated.
3. Turnaround Time (TAT) = CT - AT
4. Waiting Time (WT) = TAT - BT

Code:-

```
# -------------------- FCFS -------------------- fcfs
= sorted(processes, key=lambda x: x.at)
 current_time =
0 total_tat = 0
total_wt = 0
```

```
 gantt_fcfs = []
 print("\n========== FCFS Scheduling ==========")
print("PID\tAT\tBT\tCT\tTAT\tWT")
 for p in fcfs:     if current_time <
p.at:         current_time = p.at
     start = current_time
current_time += p.bt

    p.ct = current_time
    p.tat = p.ct - p.at
    p.wt = p.tat - p.bt
total_tat += p.tat     total_wt +=
p.wt
     gantt_fcfs.append((p.pid, start, current_time))
     print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
 print(f"\nAverage TAT: {total_tat/len(fcfs):.2f}")
print(f"Average WT: {total_wt/len(fcfs):.2f}")
```

Screenshot:-

```
========== FCFS Scheduling ==========
PID     AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      1       3       8       7       4
P3      2       8       16      14      6

Average TAT: 8.67
Average WT: 3.33
```

Task 3: SJF Scheduling

Description:-

Shortest Job First (SJF) is a scheduling algorithm in which the process with the smallest burst time is executed first. Only the processes that have already arrived are considered.

Steps:-

1. Select processes that have arrived.
2. Choose the process with the shortest burst time.
3. Execute it and calculate CT, TAT, WT
4. Repeat until all processes are completed.

Code:-

```
# -------------------- SJF (Non-Preemptive) ------------------
- sjf = processes.copy() completed = [] current_time = 0
 total_tat_sjf =
0 total_wt_sjf =
0
 gantt_sjf =
[]
 print("\n========== SJF Scheduling
==========") print("PID\tAT\tBT\tCT\tTAT\tWT")
 while len(completed) < len(sjf):     # Get available processes
available = [p for p in sjf if p.at <= current_time and p not in
completed]
     if not
available:
        current_time += 1
continue

    # Select process with shortest burst time
p = min(available, key=lambda x: x.bt)

    start = current_time
current_time += p.bt

    p.ct = current_time
    p.tat = p.ct - p.at
    p.wt = p.tat - p.bt
    total_tat_sjf += p.tat
total_wt_sjf += p.wt
     gantt_sjf.append((p.pid, start,
current_time))
```

```
    completed.append(p)
     print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
 print(f"\nAverage TAT: {total_tat_sjf/len(sjf):.2f}")
print(f"Average WT: {total_wt_sjf/len(sjf):.2f}")
```

ScreenShot

```
========== SJF Scheduling ==========
PID     AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      1       3       8       7       4
P3      2       8       16      14      6

Average TAT: 8.67
Average WT: 3.33
```

Task 4: Gantt Chart Representation

Description:-

Gantt chart is used to represent the execution order of processes over time. It helps in visualizing how CPU is allocated to different processes.

Code:-

```
# -------------------- GANTT CHART -------------------
- print("\n========== Gantt Chart (FCFS) ==========")
for p in gantt_fcfs:
    print(f"| {p[0]} ", end="")
print("|")
 for p in
gantt_fcfs:
    print(p[1], end="   ") print(gantt_fcfs[-1][2])


print("\n========== Gantt Chart (SJF) ==========") for
p in gantt_sjf:
    print(f"| {p[0]} ", end="")
print("|")
 for p in
gantt_sjf:
    print(p[1], end="   ") print(gantt_sjf[-1][2])
```

Screenshot:-

```
========== Gantt Chart (FCFS) ==========
| P1  | P2 | P3 |
0   5   8   16

========== Gantt Chart (SJF) ==========
| P1  | P2 | P3 |
0   5   8   16
```

Task 5: Performance Analysis & Comparison

Description:-

The performance of scheduling algorithms is evaluated using Average Waiting Time and Average Turnaround Time.

From the results, SJF gives better performance as it minimizes the average waiting time compared to FCFS.

Therefore, SJF is more efficient than FCFS.

Code:-

```
# -------------------- COMPARISON --------------------
print("\n========== Comparison ==========")

avg_wt_fcfs = total_wt / len(fcfs)
avg_wt_sjf = total_wt_sjf / len(sjf)
 avg_tat_fcfs = total_tat / len(fcfs)
avg_tat_sjf = total_tat_sjf /
len(sjf)

print(f"FCFS Avg WT: {avg_wt_fcfs:.2f}, Avg TAT: {avg_tat_fcfs:.2f}")
print(f"SJF Avg WT: {avg_wt_sjf:.2f}, Avg TAT: {avg_tat_sjf:.2f}")
 if avg_wt_sjf <
avg_wt_fcfs:
    print("SJF is better (less waiting time)")
else:
    print("FCFS is better")
```

Screenshot:-

```
========== Comparison ==========
FCFS Avg WT: 3.33, Avg TAT: 8.67
SJF Avg WT: 3.33, Avg TAT: 8.67
FCFS is better
```

Conclusion:-

In this assignment, we successfully implemented FCFS and SJF scheduling algorithms. We calculated scheduling parameters and compared their performance. This helped in understanding CPU scheduling concepts effectively.